

ParkingMAS

Distributed Systems Report

Bedeschi Federica
federica.bedeschi4@studio.unibo.it

September 2026

Disclaimer: AI tools were used for assistance in writing this report and developing the project code; all the generated content was reviewed and adapted by the author, who takes full responsibility for the entire final work.

Contents

1	Requirement Specification	2
1.1	Description	2
1.2	Functional Requirements	2
1.3	Non-Functional Requirements	3
2	Design	4
2.1	Architecture	4
2.2	Infrastructure	5
2.3	Inter-Agent Communication	7
2.3.1	Parking information collection	8
2.3.2	Parking search	9
2.3.3	Parking entry and exit	10
2.3.4	Parking availability updates	12
2.3.5	Communication trade-offs	13
2.4	Concurrency and Consistency	14
2.5	Fault Tolerance and Availability	15
2.6	Scalability	16
3	Implementation	17
3.1	Agent Communication	17
3.2	GUI	18
4	Testing	19
5	Deployment	20
6	Future works	21

Requirement Specification

1.1 Description

ParkingMAS is a **distributed Multi-Agent System** that models a parking system. Drivers are located in a city divided into areas, each containing several parking lots with limited spots. Drivers can autonomously move through the city and search for, enter, and exit parking lots.

1.2 Functional Requirements

1. Drivers must be able to move through the city;
2. Drivers must be able to search for available parking lots by specifying the minimum number of available spots required, and obtain the closest parking lot matching the request;
3. Drivers must be able to enter a parking lot if a parking spot is available; otherwise, they must receive a negative response;
4. Drivers must be able to exit a parking lot;
5. Drivers must be able to gracefully handle communication or operation failures and use backup plans when necessary;
6. The system must correctly handle concurrent parking requests;
7. The system must provide a GUI for observing the behavior of the simulation.

1.3 Non-Functional Requirements

1. *Concurrency*: the system must support the concurrent execution of multiple agents and operations;
2. *Distribution*: the system must support the distribution of agents across multiple execution nodes;
3. *Fault tolerance*: the system must tolerate communication failures and temporary agent unavailability without causing data loss or an inconsistent system state;
4. *Scalability*: the system must support horizontal scaling without significantly compromising system performance.

Design

2.1 Architecture

The system architecture, as shown in Figure 2.1, can be divided into two main parts: the **agents** and the **environment**, which follows the **Model-View-Controller (MVC) pattern**. The MVC consists of the **ParkingEnvironment**, which acts as the *Controller*; the **City**, which represents the *Model*; and the **CityView**, which represents the *View*. The **City** encapsulates the state of the environment and the operations that can be performed on it. The **ParkingEnvironment** acts as the *Controller* by mediating between the **City** and the **CityView**. It also mediates the interaction between the environment and the agents by providing them with percepts and allowing them to perform actions that modify the state of the **City**.

The agents are the autonomous components of the MAS, and in addition to interacting with the **ParkingEnvironment**, they interact with each other. The MAS includes three types of agents:

- **Driver Agent**: searches for a suitable parking lot by interacting with area agents, navigates to the selected parking lot, enters it, and eventually exits and returns home;
- **Area Agent**: is responsible for a specific area of the city, maintains information about the parking lots in that area, and provides parking information to driver agents;
- **Parking Agent**: represents a parking lot, handles requests from driver agents to enter or exit, and manages its available spots.

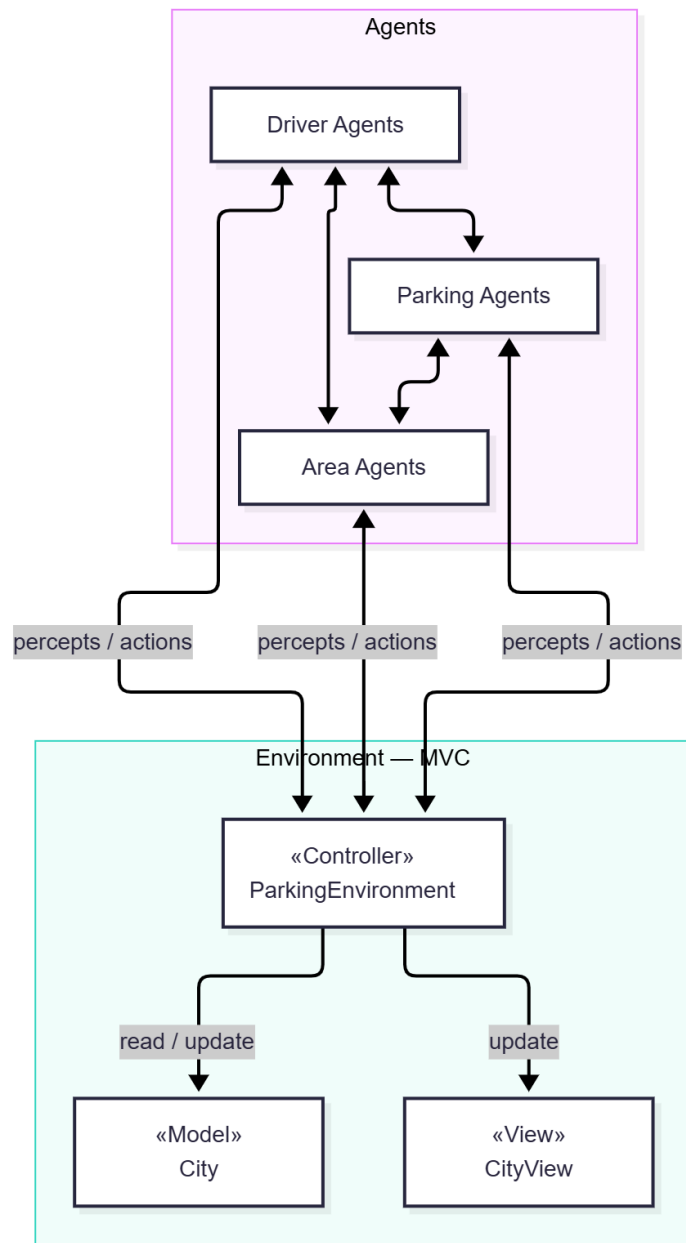


Figure 2.1: System architecture.

2.2 Infrastructure

The system infrastructure is organized into **multiple containers**, each hosting a specific set of agents, as shown in Figure 2.2. Each area of the

city is associated with a separate container hosting its `area_agent` and the `parking_agents` belonging to that area. The `driver_agents` are hosted in a dedicated container. Although driver agents could be distributed across multiple containers, a single container is sufficient in the current system since driver agents do not communicate with one another. Therefore, distributing them across multiple containers would not introduce additional communication or coordination requirements, allowing this distribution to be adopted as needed for scalability. This organization aims to separate the agents according to their **physical distribution**, allowing them to interact through the communication infrastructure as they would in a real-world distributed system.

A **Main Container** hosts the environment, including all the MVC components. In particular, the `City` maintains a single authoritative representation of the state of the environment, providing a unique source of truth. The `ParkingEnvironment`, in turn, provides the interface between the agents and the model, handling agent percepts and actions and therefore acting as the unique access point to the environment. The Main Container therefore represents the **centralized part** of the infrastructure on which the other containers depend. This centralization simplifies the management and consistency of the environment, but also makes the Main Container a potential single point of failure and bottleneck, as discussed in the following sections.

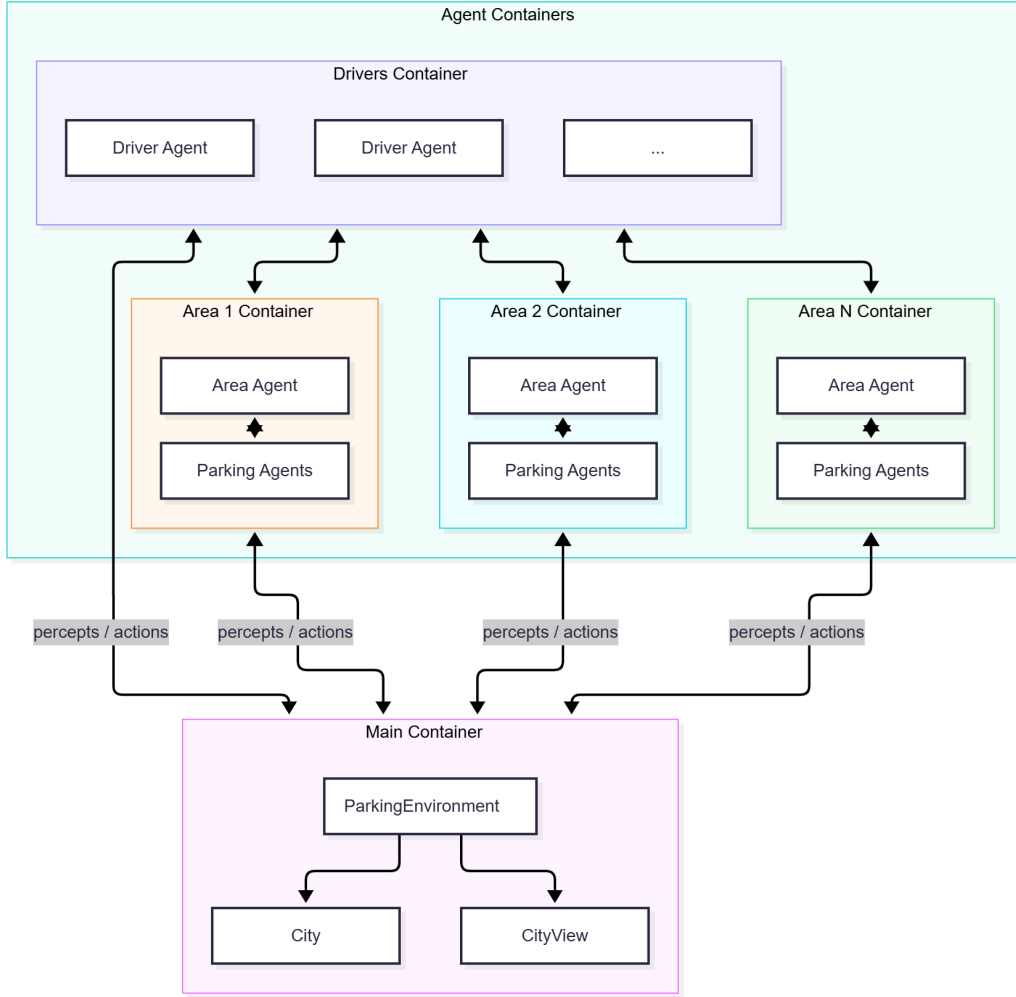


Figure 2.2: System infrastructure.

2.3 Inter-Agent Communication

The agents interact through **message-based communication**. In particular, some interactions follow a **request-response** pattern in which the requesting agent waits for a response before proceeding with the current task, while other interactions are **notifications**, in which an agent communicates a state change without requiring an immediate response from the receiver.

Communication failures are considered as part of the interaction design. Since agents may be temporarily unavailable or a request may not receive a response, request-response interactions are subject to a **bounded waiting period**. If the response is not received within the allowed time,

the requesting agent treats the interaction as a communication failure rather than waiting indefinitely. In this case, or if the requesting agent receives a communication error, it decides whether to **retry** the communication after a short delay or to proceed in a different way. The retry mechanisms are generally **bounded** in the number of failures, so that an agent does not retry indefinitely. However, some interactions intentionally use unbounded retries when the operation is necessary to maintain the agent's state or complete its current task. When the maximum number of retries is reached, the agent applies an appropriate **fallback strategy** whenever one is available. The possible outcomes can be summarized as follows:

- **successful response:** the requesting agent receives a successful response and continues its normal behavior;
- **negative response:** the request was successfully processed, but the requesting agent receives a negative response, so it follows an alternative plan;
- **communication failure:** no response was received within the allowed time, or the agent received a communication error, so it retries the communication according to its retry policy;
- **repeated communication failure:** the maximum number of retries is reached, and the agent applies a fallback strategy if one is available.

This approach provides **tolerance** to temporary communication failures, and makes each agent responsible for deciding how to react to failures encountered during its own interactions.

The following sections describe the main communication interactions in the system, and the corresponding fallback strategies.

2.3.1 Parking information collection

When an area agent starts, it collects information about the parking lots belonging to its area. As shown in Figure 2.3, using a *request-response* pattern, it requests the current information from each parking agent, which responds with the position and availability of the corresponding parking lot. The area agent then stores this information in its local state. The parking agent is the authoritative source for the state of its parking lot, while the area agent maintains a **local representation** of this information. This design allows parking searches to be performed using local information without requiring the area agent to query every parking agent for each request. In case of a

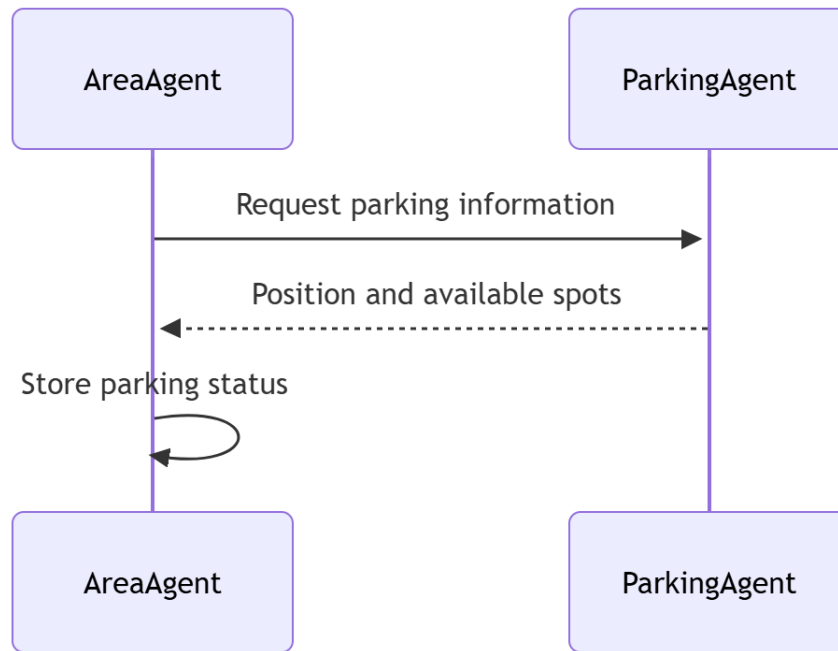


Figure 2.3: Sequence diagram illustrating the interaction between an area agent and a parking agent during the initial collection of parking information.

communication failure, the request is retried indefinitely, since the area agent needs to obtain the information about all parking lots in its area.

2.3.2 Parking search

Driver agents search for parking by communicating with area agents. As shown in Figure 2.4, a driver agent, using a *request-response* pattern, initially contacts the area agent responsible for the area in which it is located and specifies the minimum number of available spots required. The area agent searches its local parking information and returns the closest parking lot satisfying the driver's requirements. If no suitable parking lot is available in the current area, the driver receives a negative response and contacts the area agent responsible for the nearest area. If communication with the current area agent fails, the driver agent retries the request **up to three times**. If the third attempt fails, the driver agent requests parking from the area agent responsible for the nearest area. If communication with the nearest

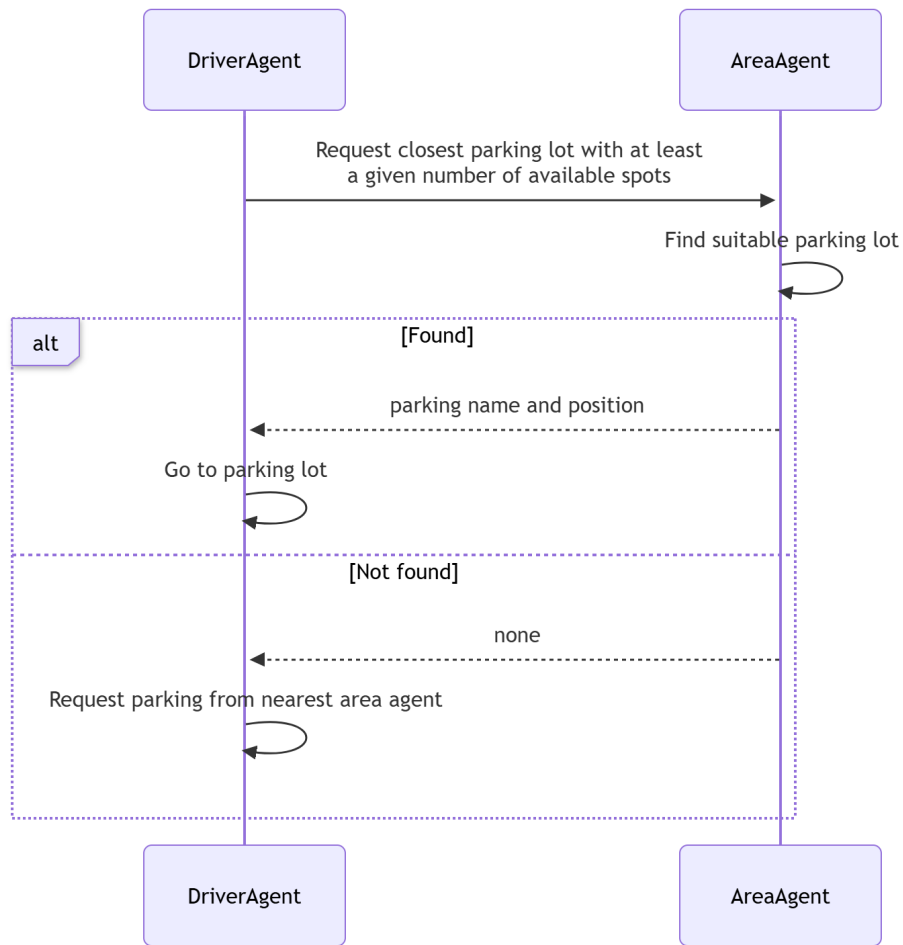


Figure 2.4: Sequence diagram illustrating the interaction between a driver agent and an area agent when requesting a parking lot.

area agent fails, or if the driver agent receives a negative response from it, the driver agent restarts the parking search protocol with the current area agent.

2.3.3 Parking entry and exit

Once a driver agent reaches the selected parking lot, it communicates with the corresponding parking agent to request **entry**, as shown in Figure 2.5. The parking agent determines whether the operation can be performed according to the current availability of the parking lot. If at least one parking

spot is available, the parking agent grants the entry request, performs the entry operation in the environment, and decreases the number of available spots. Otherwise, the request is denied, and the driver can search for another parking lot.

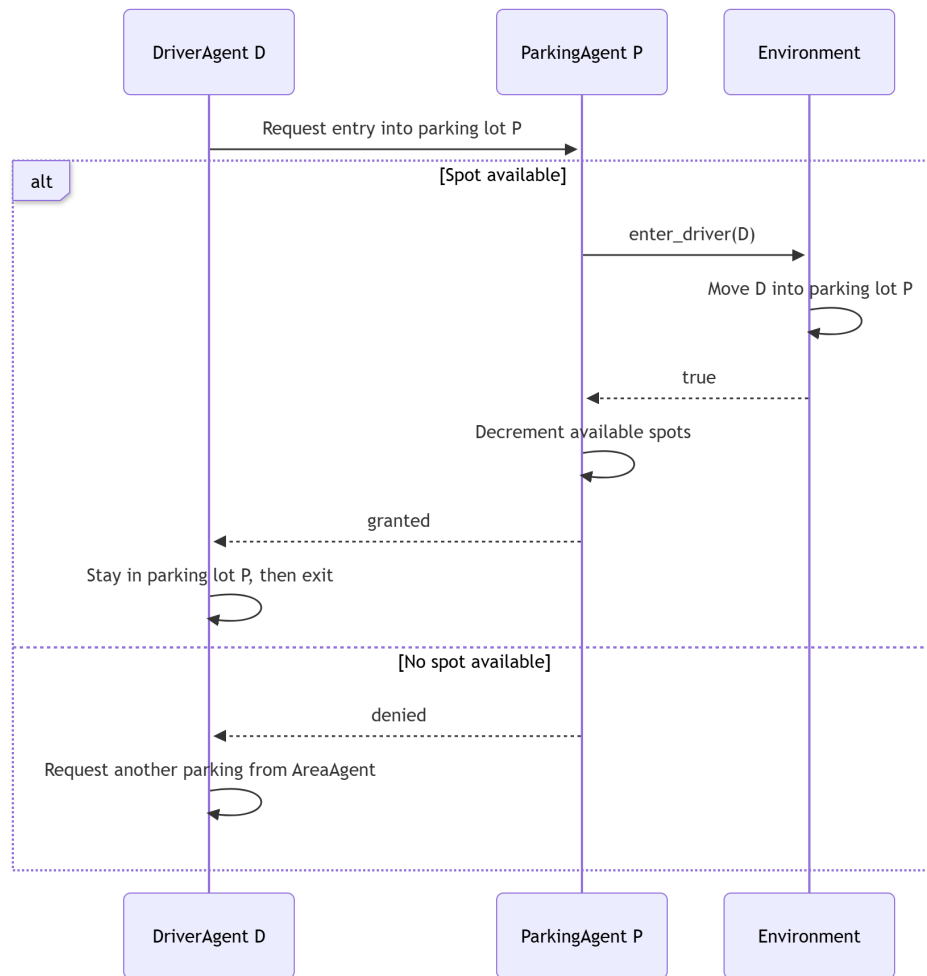


Figure 2.5: Sequence diagram illustrating the interaction between a driver agent and a parking agent when requesting to enter a parking lot.

A similar behavior occurs when a driver wants to **exit** the parking lot. In this case, the driver agent requests an exit from the parking agent, as shown in Figure 2.6. The parking agent performs the exit operation in the environment, increases the number of available spots, and confirms the exit to the driver agent, which then returns home.

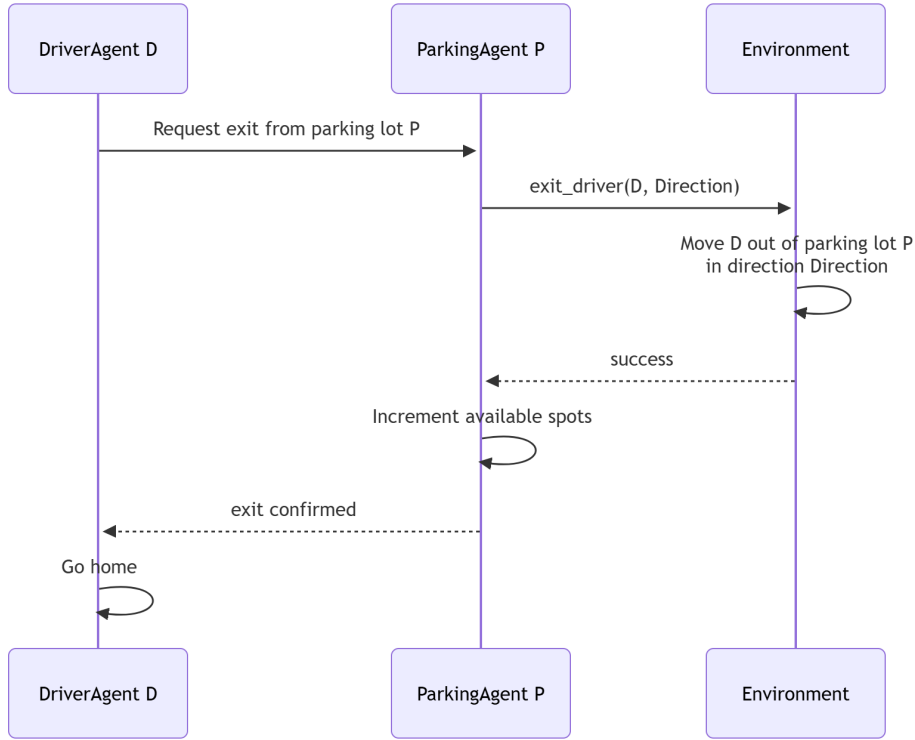


Figure 2.6: Sequence diagram illustrating the interaction between a driver agent and a parking agent when requesting to exit a parking lot.

In both scenarios, the interaction is *request-response* based. In case of a communication failure, to maintain consistency, the driver agent checks its position to determine whether the requested operation has actually been performed. If it has been performed, the driver agent can continue its behavior as if it had received a confirmation; otherwise, it can retry the communication. For entry, the driver agent retries the request up to three times. If all attempts fail, it requests another parking lot from the current area agent. For exit, instead, the driver agent retries indefinitely because it is already inside the parking lot and must eventually complete the exit operation.

2.3.4 Parking availability updates

Whenever the availability of a parking lot changes, the corresponding parking agent communicates the new state to its area agent. As shown in Figure 2.7, the parking agent notifies the area agent of the availability update, upon which the area agent updates its local status of the parking lot accordingly.

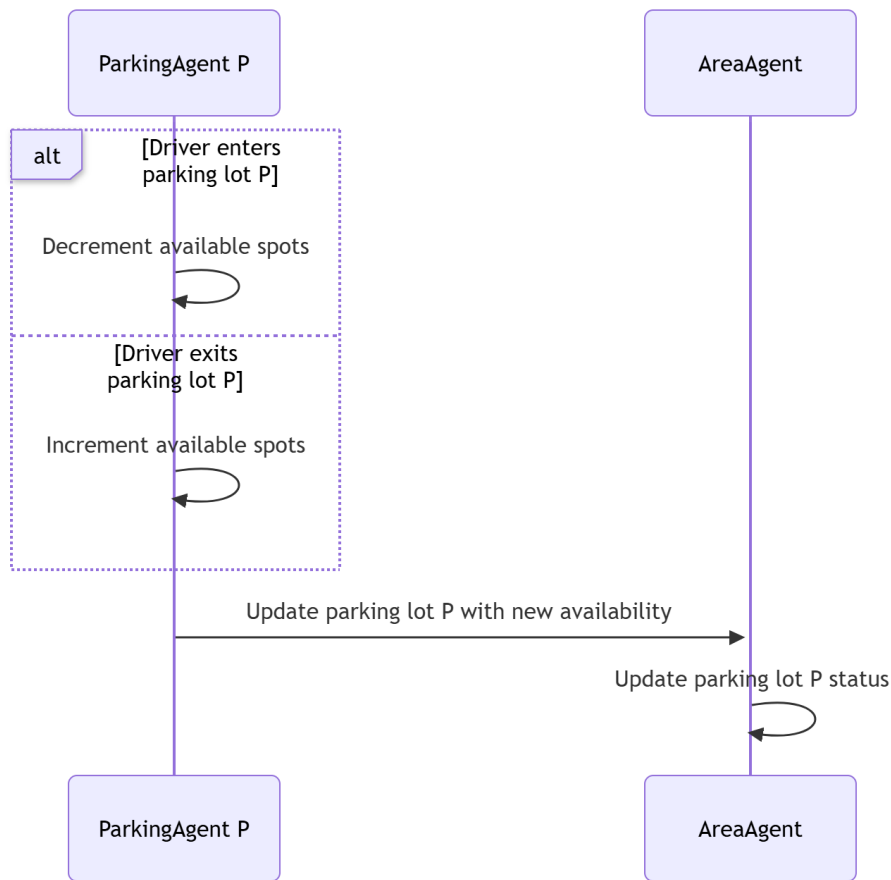


Figure 2.7: Sequence diagram illustrating the interaction between a parking agent and an area agent to update parking availability.

This communication is a *notification* rather than a request-response interaction because the parking agent does not require the area agent to return a result before continuing its operation. This update keeps the information used for parking searches up to date with changes occurring at the parking lot level. However, since the update is communicated between independent agents, the local information maintained by the area agent may temporarily differ from the authoritative state of the parking agent.

2.3.5 Communication trade-offs

The use of **local information** at the area agent level reduces communication overhead during parking searches, since the area agent does not need to

contact every parking agent for each search request. However, maintaining local copies introduces a potential consistency delay between the authoritative parking state and the information available to the area agent. This trade-off is accepted in the current design.

Request-response interactions provide a simple mechanism for obtaining the result of an operation before proceeding, but they also introduce a dependency on the availability and response time of the contacted agent. **Bounded waiting** and **retry mechanisms** mitigate this dependency by preventing an agent from remaining blocked indefinitely and allowing it to recover from temporary communication failures.

Finally, the communication model is **decentralized** at the agent level: no central agent coordinates all parking decisions. Each agent performs its own reasoning and manages the interactions required to achieve its goals. This supports agent autonomy and distribution, while requiring explicit mechanisms for coordination and communication failure handling.

2.4 Concurrency and Consistency

The system is designed to support the **concurrent execution** of multiple agents and operations. In particular, multiple driver agents may simultaneously search for parking, or request entry to or exit from the same parking lot. This introduces potential race conditions when multiple agents interact with the same shared resource. The main shared resource is the **parking availability**, which is maintained by the corresponding parking agent for each parking lot. The parking agent acts as the **authoritative source** for this state. Requests to enter or exit a parking lot are therefore handled by the corresponding parking agent, which updates its availability as part of the operation. The update is performed **atomically** with respect to other requests handled by the same parking agent, preventing concurrent entry requests from incorrectly allocating the same available spot.

The system does not maintain a single globally shared copy of parking availability among all agents. Instead, as previously mentioned, area agents maintain **local representations** of the parking information, which are updated when the corresponding parking agent changes its availability. This introduces a potential **consistency delay**, meaning that a parking search may be based on slightly outdated information.. The design therefore favors **locality and reduced communication overhead** over strong global consistency.

2.5 Fault Tolerance and Availability

The fault tolerance and availability design considers failures that may occur during **agent-to-agent communication**, such as temporary agent unavailability or messages that do not receive a response. As described in Section 2.3, request-response interactions use bounded waiting to prevent an agent from waiting indefinitely for an unavailable or unresponsive agent. Depending on the interaction, communication failures can then trigger retries, alternative actions, or fallback strategies. For example, a driver agent that cannot contact an area agent can retry the request and, after repeated failures, attempt to contact the nearest area agent. This approach provides resilience to **temporary communication failures** and allows failures affecting individual agents or containers to be partially contained without necessarily stopping the entire system. Therefore, these mechanisms allow the system to remain **available** despite temporary failures affecting individual agents or communication channels. The system does not, however, provide complete recovery from permanent failures: if an agent remains unavailable, the available fallback depends on the specific interaction and on whether an alternative agent or operation exists.

The environment is instead centralized. The `ParkingEnvironment` provides the interface through which agents perceive and modify the world, while the `City` maintains its authoritative state. Consequently, a failure of the Main Container hosting these components would prevent agents from interacting with the environment and would effectively stop the simulation. The Main Container therefore represents a single point of failure. This centralization is an intentional design choice. Replicating the environment would introduce additional consistency and coordination requirements, while the current system prioritizes a single authoritative representation of the simulated world. Fault tolerance therefore applies primarily to distributed agent communication, while the centralized environment remains a known **architectural limitation**.

Overall, in the presence of partial communication failures, the system generally prioritizes **availability** when operations can tolerate temporarily stale information, while prioritizing **consistency** for operations that modify shared parking state and therefore require the authoritative state of the corresponding parking agent.

2.6 Scalability

The system supports **horizontal scaling** at the agent level by allowing agents to be distributed across multiple containers. New areas can be introduced by deploying additional area containers, while driver agents can also be distributed across multiple containers if the number of drivers increases. As mentioned in Section 2.2, this is possible because driver agents do not communicate directly with one another, so no additional coordination is required between them.

The main limitation to scalability is the centralized environment. The `ParkingEnvironment` and the `City` constitute a centralized component through which all agents interact with the simulated world. As the number of agents increases, the centralized environment may become a **bottleneck**, since it must process an increasing number of percept and action requests while maintaining a single authoritative state. The design reduces this dependency by **limiting interactions** between the environment and the agents to operations that strictly concern the **physical aspects** of the world, such as positioning and movement. Each agent is designed to maintain its own relevant information and performs its reasoning **autonomously**. Therefore, the interactions with the environment are limited to the physical scope and are not necessary for operations that can be performed using agent-local information, such as parking requests made by driver agents to area agents.

In conclusion, the centralized environment remains a potential scalability constraint. Removing this limitation would require distributing, partitioning, or replicating the environment, which would introduce additional consistency and coordination mechanisms. This design choice is therefore accepted in favor of horizontal scalability at the agent level.

Implementation

This chapter describes the implementation of the system, focusing on the concrete mechanisms used to realize the design presented in the previous chapter. The agents are implemented in **AgentSpeak** and executed using the **Jason** agent programming framework, while **JADE** provides the underlying agent platform and communication infrastructure. Each agent type is implemented as an AgentSpeak program defining its beliefs, rules, goals, and plans. The environment and its MVC components are instead implemented in **Java**.

The system is executed using multiple **JADE containers**. As mentioned in Section 2.2, a Main Container hosts the environment, while additional containers host the area agents, parking agents, and driver agents.

3.1 Agent Communication

The communication mechanisms described in Section 2.3 are implemented using Jason’s communication primitives, which rely on JADE for the underlying agent communication infrastructure. **Request-response** interactions are implemented using an `askOne` send, specifying a timeout. For example, the `driver_agent` asks the `parking_agent` to exit from the corresponding parking lot as follows:

```
.send(P, askOne, exit(Direction), Answer, 3000);
```

In case of a communication failure, the timeout is handled by the corresponding **failure-handling plan** as follows:

```
+!process_exit_response(timeout, P) <-  
  .print("Parking agent ", P, " unavailable; retrying in  
    0.5s");  
  .wait(500);  
  !leave_parking_lot(P).
```

Notifications, instead, are implemented using an **achieve** send, as in the availability update sent by the **parking_agent** to the **area_agent**:

```
.send(Agent, achieve, update_availability(P, Spots)).
```

As far as **consistency** is concerned, as mentioned in Section 2.4, requests to enter or exit a parking lot are handled **atomically** within the corresponding **parking_agent**. This is achieved by specifying the `@[atomic]` annotation before the corresponding test-goal plans, as in:

```
@[atomic]
+?entry_status(granted)[source(D)] : available(A) & A > 0 <-
    .print("Processing entry request from driver ", D, ";
        available spots: ", A);
    enter_driver(D);
    .print("Entry granted for driver ", D, "; remaining spots
        : ", A - 1);
    -+available(A - 1).
```

This ensures that the availability check and the corresponding state update are executed atomically, preventing another concurrent intention from interleaving between them. Consequently, two or more concurrent entry requests cannot incorrectly allocate the same available spot.

At the Java level, the **City** implementation may be accessed concurrently by multiple agent actions. Its shared state is therefore managed using thread-safe data structures such as **ConcurrentHashMap**, while critical methods that modify the state are declared **synchronized**. This ensures **thread-safe** access to the shared state during concurrent operations.

3.2 GUI

The graphical interface is implemented using **Java Swing** and represents the view of the city, including the positions of drivers and parking lots. Since Swing requires GUI updates to be performed on the *Event Dispatch Thread*, they are scheduled using **SwingUtilities.invokeLater**. This ensures **thread-safe** access to the GUI components. The GUI is therefore used primarily as a monitoring mechanism, allowing the execution of the system to be observed at runtime.

Testing

The testing activity was divided into different levels to verify both the correctness of individual components and the behavior of the complete system. **Automated unit tests** were used to verify the functionality of the environment model, while **automated integration tests** were used to verify the interaction between the environment and the model. Finally, **manual end-to-end tests** were performed to evaluate the behavior of the complete distributed system, including its behavior under communication failures and concurrent operations.

Each class of the environment model is covered by **unit tests**. These tests verify the correctness of the main operations provided by the model, including driver movement, parking entry and exit, position management, and the handling of the city state. In particular, boundary conditions and invalid operations are also tested to ensure that the model does not reach an inconsistent state.

Integration tests are used to test the `ParkingEnvironment` in conjunction with the model. These tests verify the interface exposed to the agents by checking both the percepts returned by the environment and the effects of actions performed through it. This allows the consistency between the internal state of the model and the state perceived by the agents to be verified.

Finally, **manual end-to-end tests** were performed on the complete multi-agent system to evaluate its behavior during realistic execution scenarios. These tests verify the complete interaction between driver, area, and parking agents, including parking search, navigation, entry, and exit. They also cover different failure scenarios, such as temporary agent unavailability, communication timeouts, repeated communication failures, and the activation of fallback strategies.

Deployment

The distributed deployment of the system is configured through two **MAS2J** files, which together define the environment, the agents, and their execution containers. As described in Section 2.2, the configuration distinguishes between the Main Container, which hosts the environment, configured by the `parkingMain.mas2j` file, and the additional containers that host the agents, configured by the `parkingAgents.mas2j` file. In the current implementation, these containers correspond to **JADE containers**.

The system is launched through two **Gradle tasks**. The first task starts the Main Container and initializes the simulated environment. The second task starts an additional JADE container and connects it to the existing Main Container. The latter task accepts the name of the container to be started, allowing the different agent containers to be launched using the same command. Therefore, the system can be started manually by executing the Gradle tasks for the Main Container and for each required agent container. Alternatively, the provided shell script `runParkingMAS.sh` automates the startup sequence by first launching the Main Container and then starting the containers for the two configured areas (`area1` and `area2`) and for the driver agents (`drivers`). The Main Container is started before the other containers so that the JADE platform is available when they attempt to connect.

This deployment configuration allows additional area or driver containers to be started using the same mechanism, making it possible to extend the deployment without modifying the underlying agent implementation.

Future works

The main limitation of the current architecture is the **centralization of the environment**. The `ParkingEnvironment` and the `City` are hosted in the Main Container and represent a single point of failure and a potential scalability bottleneck. Several extensions could therefore be considered to improve the fault tolerance and scalability of this component, with increasing levels of architectural complexity.

A first improvement could be to containerize the JADE containers using **Docker** and configure appropriate **restart policies**. This would require adapting the deployment configuration to the containerized system, including inter-container networking, startup dependencies, and, in particular, logging and access to the graphical interface. If a container terminates unexpectedly, the container runtime could automatically restart it, reducing the duration of service interruptions caused by process failures. This approach would improve the fault tolerance of the deployment without requiring substantial changes to the application architecture. However, it would not remove the centralized nature of the environment or address the scalability bottleneck, since the environment would still be hosted by a single container.

More substantial improvements could be achieved by **replicating** or **partitioning** the environment. Replication would allow multiple environment instances to maintain copies of the simulated state, improving availability in case of failures, but would require mechanisms to keep the replicas consistent. Partitioning, instead, would divide the environment state into independently managed parts, for example according to the different areas of the city. This could distribute the workload and improve scalability, but would require additional coordination for operations involving multiple partitions. The two approaches could also be combined, with each partition having one or more replicas, providing both scalability and improved fault tolerance at the cost of greater consistency and coordination complexity.

Overall, these extensions represent different trade-offs between **simplicity, availability, fault tolerance, scalability, and coordination com-**

plexity. The current centralized design provides a simple authoritative representation of the simulated world, while progressively more distributed solutions could be introduced if the requirements for availability or scalability were to increase.